COINDCX · EVALUATION DECK

CryptoAI Trading Assistant

A full-stack, AI-augmented trading terminal that fuses real-time market data, classical indicators, machine-learning direction forecasts, and plain-English explanations — purpose-built for retail crypto traders.

React | FastAPI | MongoDB | scikit-learn | Gemini 2.5 Flash

Prepared for CoinDCX technical interview · July 2025

Sixty seconds, no slides

I built **CryptoAI** — a browser-based trading terminal that takes the guesswork out of crypto for retail users. It pulls **real-time market data** from CoinGecko, layers on **three classical indicators** (RSI, MACD, EMA), fuses them into a single **BUY/SELL/HOLD signal with confidence**, adds a **scikit-learn RandomForest** for next-bar direction, and asks **Gemini 2.5 Flash** to translate the picture into one paragraph of plain English.

On top of that sits a **risk calculator**, a **paper-trading wallet**, and a **price/signal alerts engine** — all behind JWT auth, all backed by MongoDB. ~1,900 lines, 16/16 backend tests passing, deploys in one shell command.

It is essentially a lightweight, opinionated version of the kind of Pro Dashboard CoinDCX offers today — but with an AI copilot baked in.

Retail crypto traders lose money for three reasons

1 · React to headlines

No quant discipline. Decisions driven by Twitter, not by data. Outcomes are random.

2 · No position sizing

One bad trade wipes out a month of gains. Stop-loss and R:R are folklore, not math.

3 · Indicator jargon

"RSI 72" means nothing to a new user. Charts are for experts; beginners give up or gamble.

My hypothesis

If a product fuses **signal + explanation + risk math** on one screen, activation and 30-day survival of retail traders goes up — directly aligned with CoinDCX's mission of *"making crypto accessible to every Indian"*.

One diagram, one deploy

```
React (3000) ──REST /api──▶ FastAPI (8001) ──motor──▶ MongoDB
    │                          │
    │ Recharts                 ├── httpx ──────────▶ CoinGecko  (cached 30-120 s)
    │ Tailwind                 ├── pandas/numpy ▶ RSI · MACD · EMA
    │ AuthContext              ├── scikit-learn ▶ RandomForestClassifier
    │                          └── emergentintegrations ▶ Gemini 2.5 Flash
    └── JWT (httpOnly cookie + Bearer) ── single-source auth
```

Design principles I enforced

- Single backend module · single `/api` prefix → ingress-friendly, zero deploy surprises
- **UUIDs only** — never Mongo ObjectId → JSON-safe, fork-safe
- Dumb frontend, smart backend — trading logic never leaks to the client
- Env-first config — same binary runs local, k8s, or behind a CDN

Footprint

~1.9k LINES OF CODE

16/16 BACKEND TESTS PASSING

~25 API ENDPOINTS

The signal engine — a scored vote, not a hierarchy

Indicator	Bullish	Bearish
RSI(14)	< 30 (+1)	> 70 (−1)
MACD(12,26,9)	line > signal (+1)	line < signal (−1)
EMA(20) vs EMA(50)	golden (+1)	death (−1)

Aggregate score ∈ `[−3, +3]`

- ≥ +2 → **BUY**
- −1 .. +1 → **HOLD**
- ≤ −2 → **SELL**

Why a scored vote?

Robustness. A single false positive can't overturn the other two.

It also gives a natural **confidence** number we can expose to the user ("3/3 indicators agree").

Every component writes to a `reasons[]` list — the exact input that Gemini receives for the explanation, so the LLM can never hallucinate numbers we didn't compute.

ML predictor — honest, shallow, explainable

Model

`RandomForestClassifier(n_estimators=100, max_depth=4)`

Shallow on purpose — low variance on 90-bar windows. Trained **per request** (~50 ms) so the model always sees the latest bar.

Features (9, all explainable)

- Lagged returns: t-1, t-2, t-3, t-5, t-10
- RSI(14)
- MACD − signal diff
- EMA(20) / EMA(50) ratio
- Label: 1 if next-day return > 0 else 0

Trade-off I own (and would defend)

Train-per-request trades CPU for **zero staleness**. At > 50 coins we pickle per-coin models and retrain hourly — see *Upgrade Path*, slide 10. The ML layer is a **confirmation signal**, not a standalone predictor; it never overrides the indicator vote, only colours the confidence.

LLM explanation · Caching · Security

Gemini 2.5 Flash

- Called **only** on user click → cost scales with engagement, not users
- Input = structured indicator values + signal + ML probability
- Price history is never sent → hallucination risk is bounded
- ~700 ms for a 150-token summary

Cache

- In-process TTL dict keyed by `(path, params)`
- 30 s for live price, 120 s for OHLC
- Cuts upstream CoinGecko calls by ~90 %
- Single-instance only → Redis on the roadmap

Security posture

- bcrypt w/ per-password salt
- JWT HS256 · 7-day · httpOnly cookie + Bearer
- CORS locked to `FRONTEND_URL`
- Every protected route funnels through one `get_current_user` dependency
- `{password_hash: 0}` projection on every user read

The metrics CoinDCX cares about

Metric	Why it matters to CoinDCX	This system's lever
Retail activation rate	Onboarded users who place a first trade	Paper-trading wallet removes first-trade fear
30-day survival	Users who don't blow up in month 1	Risk calculator enforces position sizing
Time-on-app	Pro-tier upsell · advertising inventory	Signal + AI explanation is a daily-return loop
Support ticket volume	"Why did I lose money?" tickets	Plain-English explanations preempt them
API cost / user	Upstream market-data bills scale with users	TTL cache cuts upstream by ~90 %

One-line value proposition

Bolt this signal + explain + risk layer onto the CoinDCX Pro chart and you turn a *chart-viewer* into a *decision-tool* — at the cost of one FastAPI service and a Redis cache.

Every trade-off has a named upgrade path

Concern	Today (MVP)	Production fix
Cache	In-process dict	Redis + pub-sub invalidation
Prices	30 s polling	Exchange WebSocket + Redis fan-out
ML	Train per request	Pickled per-coin models · nightly retrain · MLflow registry
Alerts	Manual pull (`/alerts/check`)	Celery-beat cron + Email / Telegram / Webhook adapters
Auth	JWT 7-day	Refresh-token rotation · device list · 2FA
Database	Single MongoDB	Replica set · `readPreference=secondary` for analytics
Observability	Log lines	Structured logs → Loki · Prometheus · OpenTelemetry
Market data	CoinGecko free	Exchange-native feeds (CoinDCX / Binance) via adapter protocol
Secrets	`.env`	Vault / AWS Secrets Manager · rotating JWT keys (`kid` header)

If the interviewer pushes *"why not X?"* — acknowledge the trade-off, give the upgrade path, don't apologise.

Debugging the 502 storm

Situation

Under load, `/api/market/coins` was returning 502s from CoinGecko's free tier.

Task

Keep p95 latency < 500 ms without paying for a Pro plan.

Action

- Added an in-process TTL cache keyed on `(path, params)`
- Tuned TTL per endpoint: 30 s for live price, 120 s for OHLC
- Graceful fallback — return stale cache with `{stale: true}` when upstream fails

Result

Upstream calls ↓ ~90 %. Zero 502s under the same load. User-visible latency dropped from ~900 ms p95 to **~80 ms p95**.

Making the signal explainable

Situation

V1 showed "BUY — score 2". Users didn't trust it and bounced.

Task

Make the signal **accountable** without inventing numbers.

Action

- Added a `reasons[]` list inside `compute_signal` — e.g. *"RSI 28 → oversold"*
- Fed reasons + raw numbers to Gemini with a tight prompt: `"Summarise for a retail trader in ≤ 3 sentences. Do not invent prices."`

Result

Explanation became the most-clicked element on the dashboard. **Hallucination risk is bounded** because the LLM never receives raw prices — a property I can defend in a compliance review.

First-trade fear

Situation

Beginners hesitated to register at all — "what if I lose money the first day?"

Task

Give new users a **zero-risk first interaction** that still teaches the product.

Action

- Seeded every new user with a **$10,000 virtual wallet** at registration
- Paper trades share the same signal + risk surface as real trades but store in a separate collection

Result

Activation → first interaction dropped from minutes to seconds. The same UX path doubles as an **education feature** — exactly the kind of onboarding CoinDCX uses to convert new Indian retail users.

Prepared answers

Q · Why FastAPI over Django / Flask?

Async I/O by default (CoinGecko + Gemini are I/O-bound), Pydantic contracts that double as OpenAPI docs, 2× less boilerplate.

Q · Why MongoDB over Postgres?

Schema evolves weekly at MVP stage. No joins in the read path. UUID ids. ETL nightly into Postgres when analytics matures.

Q · How do you prevent ML over-fitting?

max_depth = 4 · 100 trees · 90-bar window. Per-request retrain keeps the model honest. CV is the next step.

Q · Switch to CoinDCX market data — how?

Introduce a `MarketDataProvider` protocol with `get_coins` / `get_ohlc` / `get_price`. Ship CoinDCXProvider alongside CoinGeckoProvider. Zero change below the adapter — indicators only consume a `pandas.Series` of closes.

Q · Handle LLM cost + latency?

Gemini is called only on explicit user intent (Explain button). Cache explanation by `(coin, signal_bucket, day)` → repeat users in the session get it free.

Q · Biggest weakness today?

Single-instance in-memory cache — breaks on horizontal scale. Fix: Redis + per-coin invalidation pub-sub. ~1 day of work.

Q · First sprint at CoinDCX — what ships?

(1) Exchange-native WebSocket feed + Redis fan-out. (2) Pickled ML models + nightly retrain. (3) Email/Telegram alert delivery — the feature that makes the app survive being closed in a tab.

Q · Regulatory overlay (SEBI / FIU-IND)?

KYC before paper→real · append-only trade log with hash chain · data residency in India. All infra-layer, no change to the signal engine.

CLOSING · THREE THINGS TO LEAVE THEM WITH

Opinionated.
Named upgrade paths.
Swap-one-adapter to CoinDCX.

1 · Opinionated

It has a view about what retail traders need, and the architecture reflects that view.

2 · Nothing accidental

Every trade-off has a named upgrade path. Nothing is a dead end.

3 · Ready for CoinDCX

Adapting to the CoinDCX stack is swap-one-adapter work, not a rewrite. That's by design.

Thank you. Questions?